

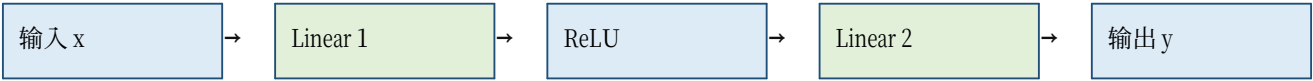
第 12 讲

第一次完整 NumPy 神经网络

不用 PyTorch，把 Forward 从头写出来

本讲定位	这是第二阶段“从神经元到 MLP”的收口课。我们不引入 Loss、不训练参数，也不使用 PyTorch。目标只有一个：你能够独立写出一个两层 MLP 的完整 Forward，并知道每一行代码在算什么。
必须掌握	参数如何组织；Linear 和 ReLU 如何组合；单样本与 Batch 的 Shape 如何变化；参数数量如何计算；Forward 与 Training 的区别；为什么这就是 Transformer FFN 的最小原型。

1. 先把前 11 讲压缩成一条计算链



到目前为止，我们分别学过 Weight、Bias、矩阵乘法、Activation、MLP、Forward Propagation 和 Computational Graph。第 12 讲不再增加新的数学组件，而是把它们真正装到同一个程序里。

$$\begin{aligned} z &= xW + b \\ a &= \text{ReLU}(z) \\ y &= aW + b \end{aligned}$$

一句话理解	一个完整的前向传播，就是“拿当前参数，把输入一步一步算成输出”。此时参数不会自己改变。
-------	---

你以前在 LLM 课程里见过什么？

你以前见过 Transformer FFN / MLP。现在我们从最底层重建它：FFN 不是一个神秘模块，而是 Linear - Activation - Linear 的组合。

2. 我们今天搭建什么网络？

为了能手算，我们用一个很小的两层 MLP：输入维度 3，隐藏维度 4，输出维度 2。

$$3 \rightarrow 4 \rightarrow 2$$

对象	Shape	含义
x	(3,)	一个样本，有 3 个输入特征
W ₁	(3,4)	把 3 维输入映射为 4 维隐藏表示
b ₁	(4,)	4 个隐藏神经元各有一个 bias
z ₁	(4,)	第一层 Linear 的输出，尚未激活
a ₁	(4,)	ReLU 之后的隐藏表示
W ₂	(4,2)	把 4 维隐藏表示映射为 2 维输出
b ₂	(2,)	2 个输出各有一个 bias
y	(2,)	最终输出

Shape 规则	Linear 主要改变最后一个特征维。3 维经过第一层变成 4 维；ReLU 不改变 Shape；第二层再把 4 维变成 2 维。
----------	---

3. 第一步：明确参数是什么

我们直接给定一组参数。注意：这不是训练得到的参数，只是为了演示 Forward 而人工指定。

```
W1 = np.array([
    [ 0.5, -1.0,  0.0,  1.0],
    [ 1.0,  0.5, -0.5,  0.0],
    [-0.5,  1.0,  1.0,  0.5],
])
b1 = np.array([0.1, -0.2, 0.3, 0.0])

W2 = np.array([
    [ 1.0, -0.5],
    [ 0.5,  1.0],
    [-1.0,  0.5],
    [ 0.2,  0.8],
])
b2 = np.array([0.1, -0.2])
```

为什么 W_1 是 3×4 ？

因为输入有 3 个特征，而我们希望产生 4 个隐藏神经元的输出。每个隐藏神经元都需要一组长度为 3 的 Weight。把 4 组 Weight 并排放起来，就是一个 3×4 的矩阵。

参数量怎么算？

$$3 \times 4 + 4 + 4 \times 2 + 2 = 26$$

科研阅读连接	论文中的 hidden size、intermediate size 一旦给出，你就可以用同样的方法估算 MLP/FFN 参数量。
--------	---

4. 第二步：写出最小 Linear 和 ReLU

我们暂时不使用任何深度学习框架，只用 NumPy。

```
def linear(x, weight, bias):  
    return x @ weight + bias
```

```
def relu(x):  
    return np.maximum(0.0, x)
```

第一行到底做了什么？

`x @ weight` 是矩阵乘法/向量-矩阵乘法，用 `Weight` 重组特征；随后 `+ bias` 给每个输出维度加上自己的偏置。

第二个函数为什么这么短？

因为 ReLU 是逐元素函数：每个数字单独执行 `max(0, x)`。它不需要知道其他位置的数字，也不改变 Shape。

重要区别	Linear 会把多个输入特征混合到新的特征里；ReLU 通常不混合位置，只对每个位置的数值做非线性变换。
------	---

5. 第三步：完整写出 forward()

```
def forward(x, W1, b1, W2, b2):  
    z1 = linear(x, W1, b1)    # (3,) -> (4,)  
    a1 = relu(z1)             # (4,) -> (4,)  
    y  = linear(a1, W2, b2)   # (4,) -> (2,)  
    return y
```

这 3 行代码，就是一个两层 MLP 的 Forward。每个变量都有明确角色：

- z_1 ：第一层 Linear 的 pre-activation，也就是“进入激活函数之前”的值。
- a_1 ：经过 ReLU 之后真正送入下一层的隐藏表示。
- y ：最终输出。此时它只是数值，不一定是概率。

常见误区

看到最终输出 $y=[3.3, -1.35]$ ，不要自动把它叫“概率”。除非后面明确经过 Sigmoid/Softmax 等概率映射，否则它只是模型输出分数/连续值。

6. 用一个样本完整手算

$$\mathbf{x} = [1, 2, -1]$$

第一层计算：

$$\mathbf{z} = \mathbf{x}\mathbf{W} + \mathbf{b} = [3.1, -1.2, -1.7, 0.5]$$

其中第一个隐藏神经元可以单独拆成：

$$1 \times 0.5 + 2 \times 1.0 + (-1) \times (-0.5) + 0.1 = 3.1$$

再经过 ReLU：

$$\mathbf{a} = \text{ReLU}(\mathbf{z}) = [3.1, 0, 0, 0.5]$$

第二层计算：

$$\mathbf{y} = \mathbf{a}\mathbf{W} + \mathbf{b} = [3.3, -1.35]$$

你应该看到的结构

网络不是“整个黑盒一次算完”。它只是在不断生成新的中间 Tensor。每一步都能写出输入、规则、输出和 Shape。

7. Forward 的真正含义：当前参数下的一次函数求值

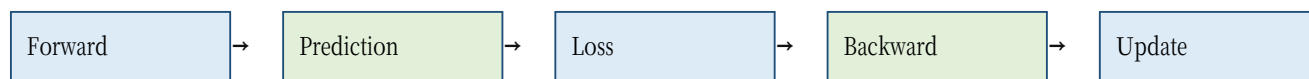
当 W_1 、 b_1 、 W_2 、 b_2 固定以后，这个网络就是一个确定的函数 $f(x)$ 。给它一个输入，就得到一个输出。

$$y = f(x; W, b, W, b)$$

分号只是强调： x 是数据， W/b 是参数。现在我们只是“使用参数”，还没有“学习参数”。

Forward $\neq$ Training	仅仅调用 forward() 不会让 Weight 变好。训练至少还需要 Target、Loss、Gradient、Backpropagation、Optimizer，以及用梯度修改参数。
-------------------------	--

完整训练闭环现在还缺什么？



目前我们只掌握了最左边的 Forward。后面的课程会逐步补齐。

8. Batch：一次让多个样本经过同一个网络

如果一次放入 3 个样本，每个样本 3 个特征：

$$\mathbf{X}.\text{shape} = (3,3)$$

阶段	运算	Shape
输入	$\mathbf{X}$	(3,3)
第一层	$\mathbf{X} @ \mathbf{W}_1 + \mathbf{b}_1$	(3,4)
ReLU	$\text{ReLU}(\mathbf{z}_1)$	(3,4)
第二层	$\mathbf{a}_1 @ \mathbf{W}_2 + \mathbf{b}_2$	(3,2)

三个样本共享同一套 $\mathbf{W}_1$ 、 $\mathbf{b}_1$ 、 $\mathbf{W}_2$ 、 $\mathbf{b}_2$ 。也就是说，Batch 并不是建立了 3 个不同模型，而是用同一个模型同时处理 3 条数据。

实验结果	程序验证 Batch Forward 与 “把 3 个样本逐个送入同一网络再 stack” 完全一致，最大绝对误差为 0。
------	---

9. 为什么要保存中间变量？

我们的实验函数除了返回 y ，还返回 z_1 和 a_1 。原因不是为了麻烦，而是因为后面 Backpropagation 会需要这些中间信息。

$$\mathbf{x} \rightarrow \mathbf{z} \rightarrow \mathbf{a} \rightarrow \mathbf{y}$$

计算图中的每个节点都记录“前面发生过什么”。以后求梯度时，我们会反过来问：

- Loss 对 y 有多敏感？
- y 对 a_1 有多敏感？
- a_1 对 z_1 有多敏感？
- z_1 对 W_1 / x 有多敏感？

提前建立直觉

Forward 负责产生中间值；Backward 会利用计算关系把“输出端的误差影响”逐层传回参数。

10. 参数扰动实验：只改一个 Weight，会发生什么？

实验中，我们只把 $W_1 [0,0]$ 增加 1，其余参数完全不动。

量	修改前	修改后	变化
z_1	[3.1,-1.2,-1.7,0.5]	[4.1,-1.2,-1.7,0.5]	[1,0,0,0]
y	[3.3,-1.35]	[4.3,-1.85]	[1,-0.5]

为什么只有第一个隐藏神经元先发生变化？因为我们只改了“输入特征 0 -> 隐藏神经元 0”的 Weight。随后，这个隐藏神经元又通过 W_2 同时影响两个输出，所以最终两个输出都变化了。

这就是依赖传播	一个参数不会凭空影响全网络。影响沿 Computational Graph 的连接方向传播。Backpropagation 以后只是把“影响程度”用导数定量化。
---------	--

11. 从本讲 MLP 到 Transformer FFN

我们今天搭的网络：

$$\mathbf{x} \rightarrow \text{Linear}(\mathbf{D}, \mathbf{H}) \rightarrow \text{ReLU} \rightarrow \text{Linear}(\mathbf{H}, \mathbf{O}) \rightarrow \mathbf{y}$$

Transformer 中的 FFN / MLP 通常也是同一思想：

$$(\mathbf{B}, \mathbf{T}, \mathbf{D}) \rightarrow \text{Linear}(\mathbf{D}, \mathbf{H}) \rightarrow \text{GELU/SiLU/SwiGLU} \rightarrow \text{Linear}(\mathbf{H}, \mathbf{D}) \rightarrow (\mathbf{B}, \mathbf{T}, \mathbf{D})$$

不同点只是规模、Activation 和具体结构更复杂；底层组件没有换。

LLM 连接	以后看到 Transformer Block 里的 FFN，你应该能够把方框翻译成“对每个 Token 的最后一维执行一组 Linear + 非线性 + Linear”。
--------	---

12. 一次性看懂完整 NumPy 代码

```
import numpy as np

def linear(x, weight, bias):
    return x @ weight + bias

def relu(x):
    return np.maximum(0.0, x)

def forward(x, W1, b1, W2, b2):
    z1 = linear(x, W1, b1)
    a1 = relu(z1)
    y = linear(a1, W2, b2)
    return y

x = np.array([1.0, 2.0, -1.0])
y = forward(x, W1, b1, W2, b2)
print(y)
```

对初学者来说，这段代码真正重要的不是“只有十几行”，而是你现在已经能解释其中每一行背后的数学。

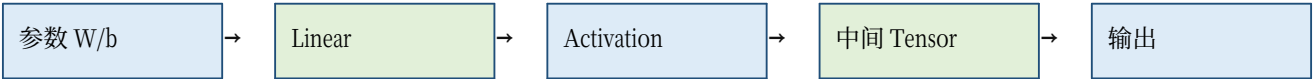
最低闭环能力

以后即使忘了某个框架 API，只要你还理解 Tensor、矩阵乘法、Weight/Bias、Activation 和 Forward，就能重新把最基本的网络搭出来。

13. 常见误区

误区 1	“我写了 forward，所以网络已经训练了。” 错。Forward 只是用当前参数计算输出。
误区 2	“最终输出一定是概率。” 错。它可能是 logit、回归值、特征向量或其他分数。
误区 3	“一个 Batch 有多套参数。” 错。通常是同一套参数处理不同样本。
误区 4	“ReLU 只是为了把负数删掉。” 不完整。更核心的作用是引入非线性，使多层网络不能被整体折叠成一个 affine。
误区 5	“隐藏神经元就是可直接解释的人类概念。” 不应默认。单个神经元的语义通常依赖训练、上下文和整个网络。

14. 本讲知识地图



你现在应该会	验收标准
自己写 Linear	能解释 $x @ W + b$ 的每一步和 Shape
自己写 ReLU	知道逐元素改变数值但不改变 Shape
自己写两层 MLP Forward	不用 PyTorch 也能得到输出
计算参数量	能算 3->4->2 为 26 个参数
处理 Batch	知道 Batch 共享参数
区分 Forward 和 Training	知道还缺 Loss/Backward/Optimizer
连接 Transformer FFN	看到 FFN 不再当黑盒

15. 与下一阶段的衔接

现在我们已经能让一个网络输出预测，但它还存在一个根本问题：

问题	网络输出了 [3.3, -1.35]，我们怎么知道它“好不好”？离正确答案差多少？
----	---

接下来第三阶段会从这个问题的出发，引入函数、斜率、导数以及 Loss。真正的训练闭环将开始形成。

Prediction + Target → Loss

其中在正式讲 Loss 之前，我们会先补足理解梯度需要的最小数学基础。

下一讲	第 13 讲：函数、斜率与导数的最小数学基础。目标不是学完整微积分，而是让“一个参数改变一点，输出会怎么变”变得可计算。
-----	--

16. 本讲总结

- 一个两层 MLP 可以写成： $z_1 = xW_1 + b_1$ ， $a_1 = \text{ReLU}(z_1)$ ， $y = a_1 W_2 + b_2$ 。
- Linear 负责混合特征，Activation 负责引入非线性。
- Forward 是“在当前参数下计算输出”，不是训练。
- 单样本和 Batch 使用同一套参数，只是 Shape 多了 Batch 维。
- 中间变量不是多余的；它们会成为后面 Backpropagation 的路径节点。
- 今天这个小 MLP 就是 Transformer FFN 的底层原型。

本讲真正掌握的标志	给你一个任意小型 MLP 的参数 Shape，你可以独立判断每一层输入输出 Shape、参数量，并用 NumPy 写出 Forward。
-----------	--